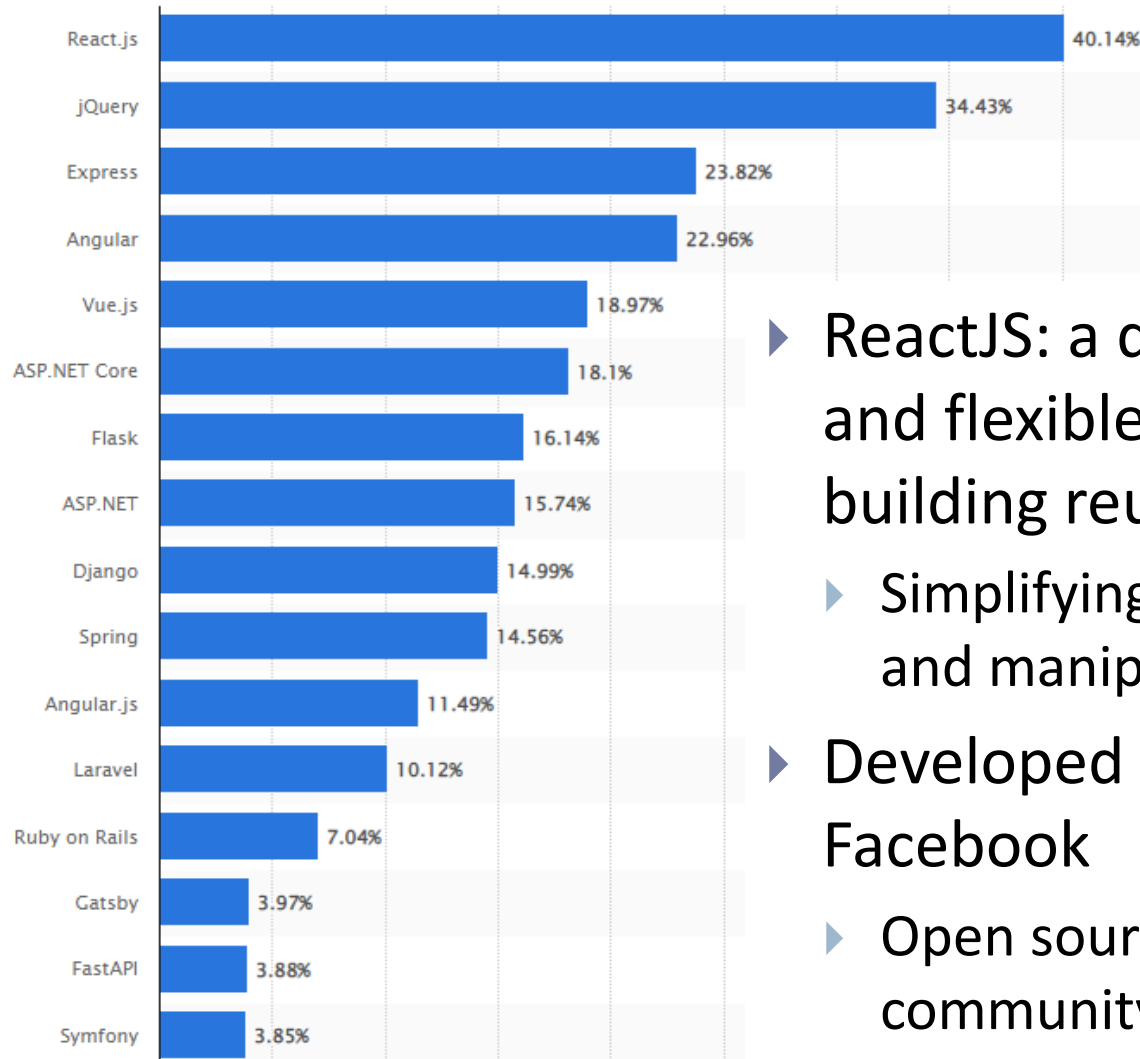
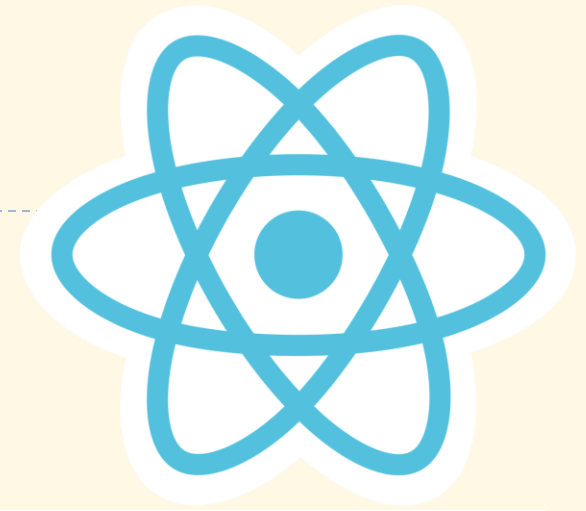

ReactJS

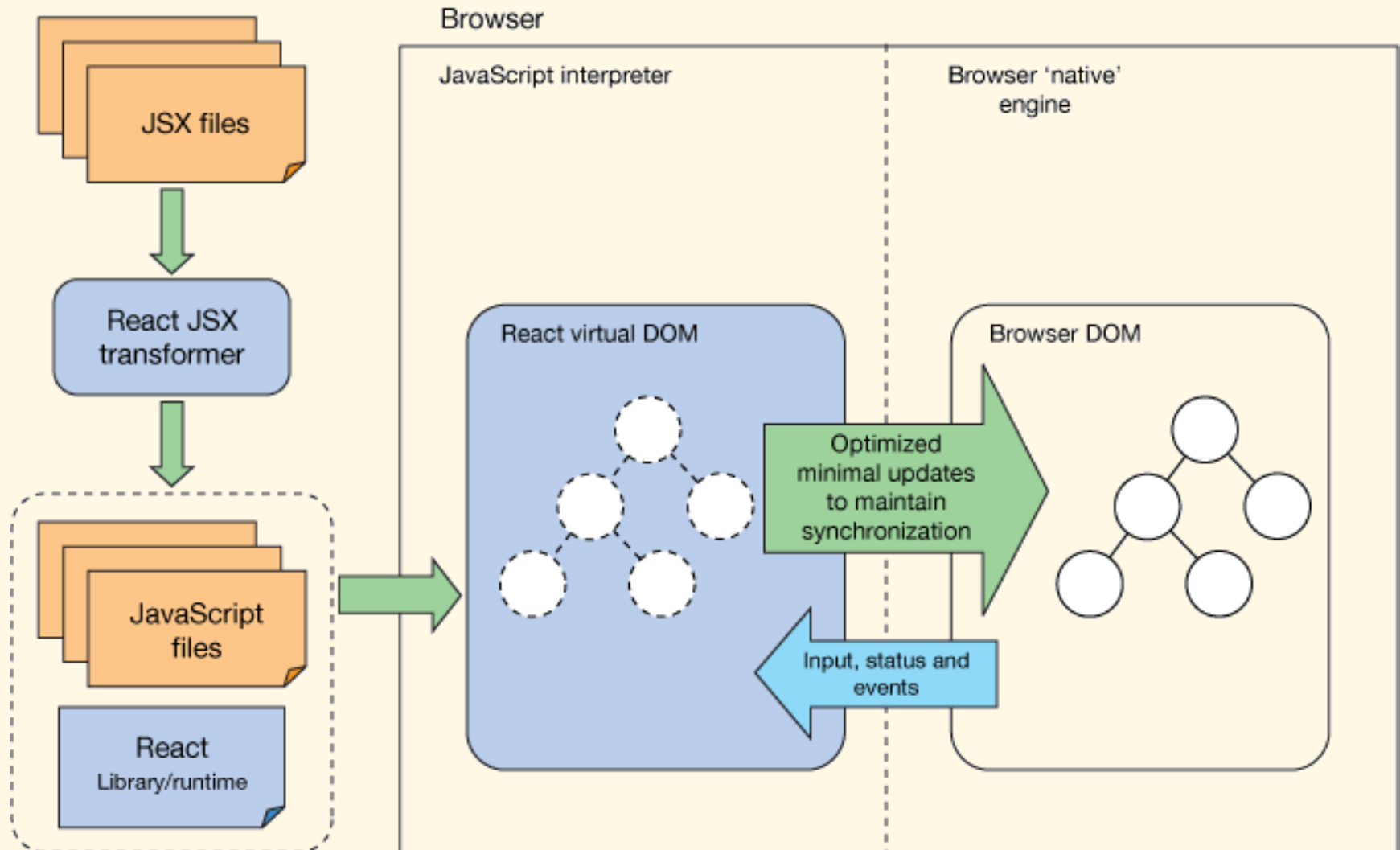


Introduction



- ▶ ReactJS: a declarative, efficient, and flexible JavaScript library for building reusable UI components
 - ▶ Simplifying the DOM management and manipulation
- ▶ Developed and maintained by Facebook
 - ▶ Open source, contributed by the community

React Operations



Setup

- ▶ Create a React project:

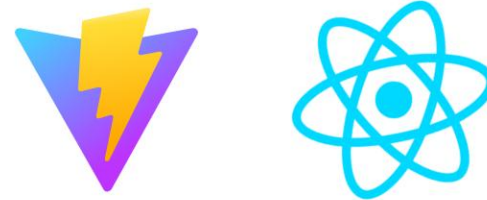
- ▶ `npm create vite@latest <project-name>`
- ▶ `npm install`

- ▶ Deploy development web:

- ▶ `npm run dev`

- ▶ Build production web:

- ▶ `npm run build`



Vite + React

Components and DOM

- ▶ JSX (JavaScript Extension) language
- ▶ Built-in components:
 - ▶ `div, a, p, img, ...`
- ▶ Associate React to a DOM element
 - ▶

```
ReactDOM.render(  
  <App />,  
  document.getElementById('root')  
) ;
```

Custom Components

- ▶ Custom class components:

- ▶ `class MyComponent extends React.Component {
 render() {
 return <div>Hello</div>
 }
}`

- ▶ Custom function components:

- ▶ `function MyComponent() {
 return <div>Hello</div>
}`

- ▶ Important: Custom component must be Capitalized

Attributes and Text

- ▶ As string literals:

- ▶ `<a className="link" href="...">Open</a>`

- ▶ As expressions:

- ▶ `
{title}`

Element in Variable

```
▶ function SampleComponent() {  
    const list = <ul className="list">  
        ...  
    </ul>;  
  
    return <div>{list}</div>;  
}
```


Inline Styling

```
▶ const myStyle = {  
    fontSize: 80,  
    fontFamily: "Courier",  
    color: "#003300"  
}
```

```
function MyComponent() {  
    return <h1 style={myStyle}>Head</h1>  
}
```

Imported Styling

► `import "../styles.css";`

```
function MyComponent() {  
    return <h1 className="header">Head</h1>  
}
```

Properties (Props) in Class Components

- ▶

```
class MyButton extends Component {  
  render() {  
    return <button className="my-button">  
      {this.props.title}  
    </button>;  
  }  
}
```
- ▶

```
const form = <div>  
  <MyButton title="Button 1" />  
  <MyButton title="Button 2" />  
</div>
```

Properties (Props) in Function Components

- ▶

```
function MyButton(props) {  
  return <button className="my-button">  
    {props.title}  
  </button>;  
}
```
- ▶

```
const form = <div>  
  <MyButton title="Button 1" />  
  <MyButton title="Button 2" />  
</div>
```

Composition

- ▶ Use normal props and the special `children` prop to pass children elements

- ▶ Example:

```
function Layout(props) {  
  return <div className="page">  
    <div className="header">{props.header}</div>  
    <div className="body">{props.children}</div>  
    <div className="footer">{props.footer}</div>  
  </div>  
}
```

```
function App() {  
  return <Layout  
    header={<h1>Title</h1>  
    footer={<div>Footer</div>  
  >  
    <p>Hello</p>  
    <p>Content</p>  
  </Layout>  
}
```

Exercise

- ▶ Create a page in React showing a blog post and readers' comments

Andrew Jones Blog

Hi, my name is Andrew Jones and I am the head of web development in our company. I decided to start this blog in order to share the most interesting remarks and ideas that I come across during my day-to-day work. I will share all sorts of interesting information related to activities of our company and to web development in general. I believe that it will be interesting reading for all our customers, partners and all other individuals interested in web development. And of course, you can post comments on each blog post in case that you want to share your opinion, have something to add or if you want to raise a discussion related to a post's topic.

Blog post name:



[Remote Management](#)

In this blog post, I will share some remarks regarding communication between our former New York Office and the newly setup London Office.

Andrew Jones | 3/23/2011 3:12:26 PM | [2 comments](#)



[Expanding to Europe](#)

In this blog post, I will try to share some of my impressions of the recent expansion of our operations to the Old Continent.

Andrew Jones | 3/21/2011 5:57:47 PM | [2 comments](#)

Using a custom
component

States in Class Components

```
▶ class Person extends Component {  
  constructor(props) {  
    super(props)  
  
    this.state = {  
      name: "John Doe",  
      age: 20  
    }  
  }  
  
  render() {  
    return <div>  
      {`${this.state.name} is ${this.state.age} years old`}  
    </div>;  
  }  
}
```

Updating States

```
▶ class Person extends Component {  
    // ...  
    setName(newName) {  
        this.setState({  
            ... this.state,  
            name: newName  
        });  
    }  
}
```

▶ Immutability is important to detect DOM changes! It's incorrect to do:

```
▶ this.state.name = newName;
```

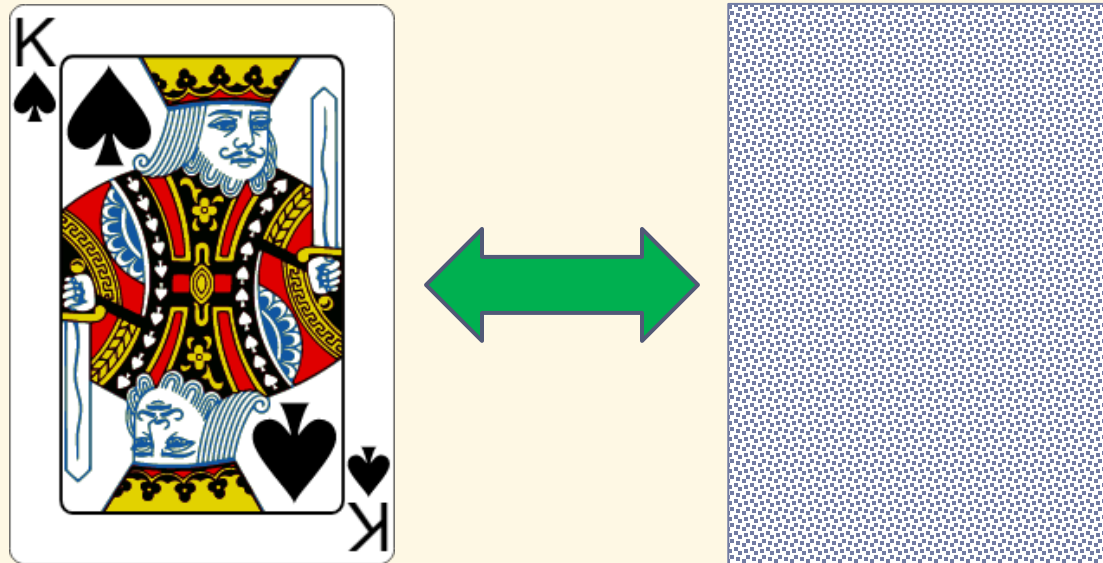

States in Function Components

```
▶ import { useState } from "react";
```

```
function Person() {  
  const [name, setName] = useState("John Doe");  
  const [age, setAge] = useState(20);  
  
  return <div>`${name} is ${age} years old`</div>;  
}
```

Exercise

1. Create an image that changes every second
2. Create a card that automatically flips up and down every 2 seconds
 - ▶ Use CSS styling, props, states
 - ▶ Do it with both class and function components



Event Handling: Class Components

```
▶ class MyButton extends Component {  
  handleClick() {  
    console.log("Button clicked")  
  }  
  
  render() {  
    return <button onClick={this.handleClick}>  
      Hello  
    </button>  
  }  
}
```

Binding a Function to an Element

- ▶ If a function needs access the parent component, we need to bind it to the component instance

- ▶ Counter-example:

```
class MyButton extends Component {
  constructor(props) {
    super(props)
    this.state = { clicked: false }
  }

  handleClick() {
    this.setState({ clicked: true }); // Error
  }

  render() {
    return <button onClick={this.handleClick}>
      Hello</button>
    }
  }
}
```

Method 1

```
▶ class MyButton extends Component {  
  constructor(props) {  
    super(props)  
    this.state = { clicked: false }  
    this.handleClick = this.handleClick.bind(this)  
  }  
  
  handleClick() {  
    this.setState({ clicked: true }); // Fine  
  }  
  
  render() {  
    return <button onClick={this.handleClick}>  
      Hello</button>  
    }  
  }  
}
```

Method 2

```
▶ class MyButton extends Component {  
  constructor(props) {  
    super(props)  
    this.state = { clicked: false }  
  }  
  
  handleClick() {  
    this.setState({ clicked: true }); // Fine  
  }  
  
  render() {  
    return <button onClick=  
      {this.handleClick.bind(this)}>  
      Hello</button>  
    }  
  }  
}
```

Method 3

- ▶

```
class MyButton extends Component {  
  constructor(props) {  
    super(props)  
    this.state = { clicked: false }  
  }  
  
  handleClick() {  
    this.setState({ clicked: true }); // Fine  
  }  
  
  render() {  
    return <button onClick=  
      { () => this.handleClick() }>  
      Hello</button>  
  }  
}
```
- ▶ Methods 2, 3 create a new function each time the component renders, which may have performance implications → Method 1 is preferred

Passing Parameters to an Event Handler

- ▶ Sometimes, we want to pass an extra parameter to an event handler
- ▶ Only possible with Methods 2 and 3
 - ▶ `<button onClick={ (e) => this.deleteRow(id, e) }>Delete Row</button>`
 - ▶ `<button onClick={this.deleteRow.bind(this, id) }>Delete Row</button>`

Event Handling: Function Components

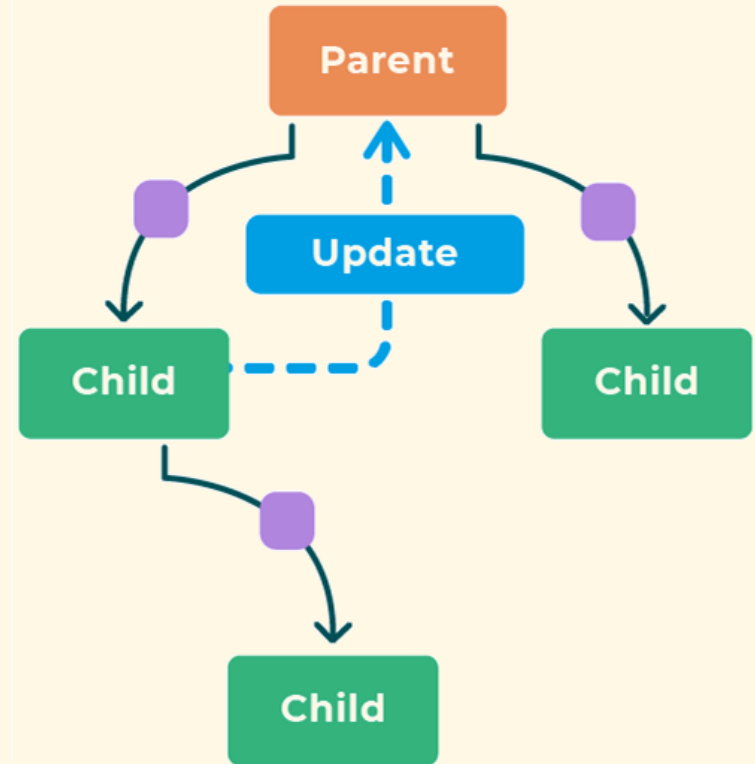
```
▶ function MyButton() {  
    const [clicked, setClicked] = useState(false);  
  
    const handleClick = () => {  
        setClicked(true)  
    }  
  
    return <button onClick={handleClick}>  
        Hello  
    </button>  
}
```

Exercise

- ▶ Change the card component to make it flips in response to mouse click events

Lifting States Up

- ▶ When multiple components rely on the same changing data, lift the shared states up to their closest common ancestor
 - ▶ Ancestors have access to their children's states
 - ▶ It's simpler



Example

```
function MyValue({text}) {
  return <span className="text">{text}</span>
}

function MyButton({title, handler}) {
  return <button onClick={handler}>{title}</button>
}

function IntPicker() {
  const [value, setValue] = useState(10);
  return <>
    <MyValue text={value} />
    <MyButton title="+"
      handler={() => setValue(value + 1)} />
    <MyButton title="-"
      handler={() => setValue(value - 1)} />
  </>
}
```

Exercise

- ▶ Create a BMI calculator as in the figure.
 - ▶ $BMI = w/h^2$ (w in kg, h in meters)
 - ▶ < 18.5 : underweight
 - ▶ $18.5 \div 25$: normal
 - ▶ $25 \div 30$: overweight
 - ▶ ≥ 30 : obese

BMI Calculator

Enter weight (kg):

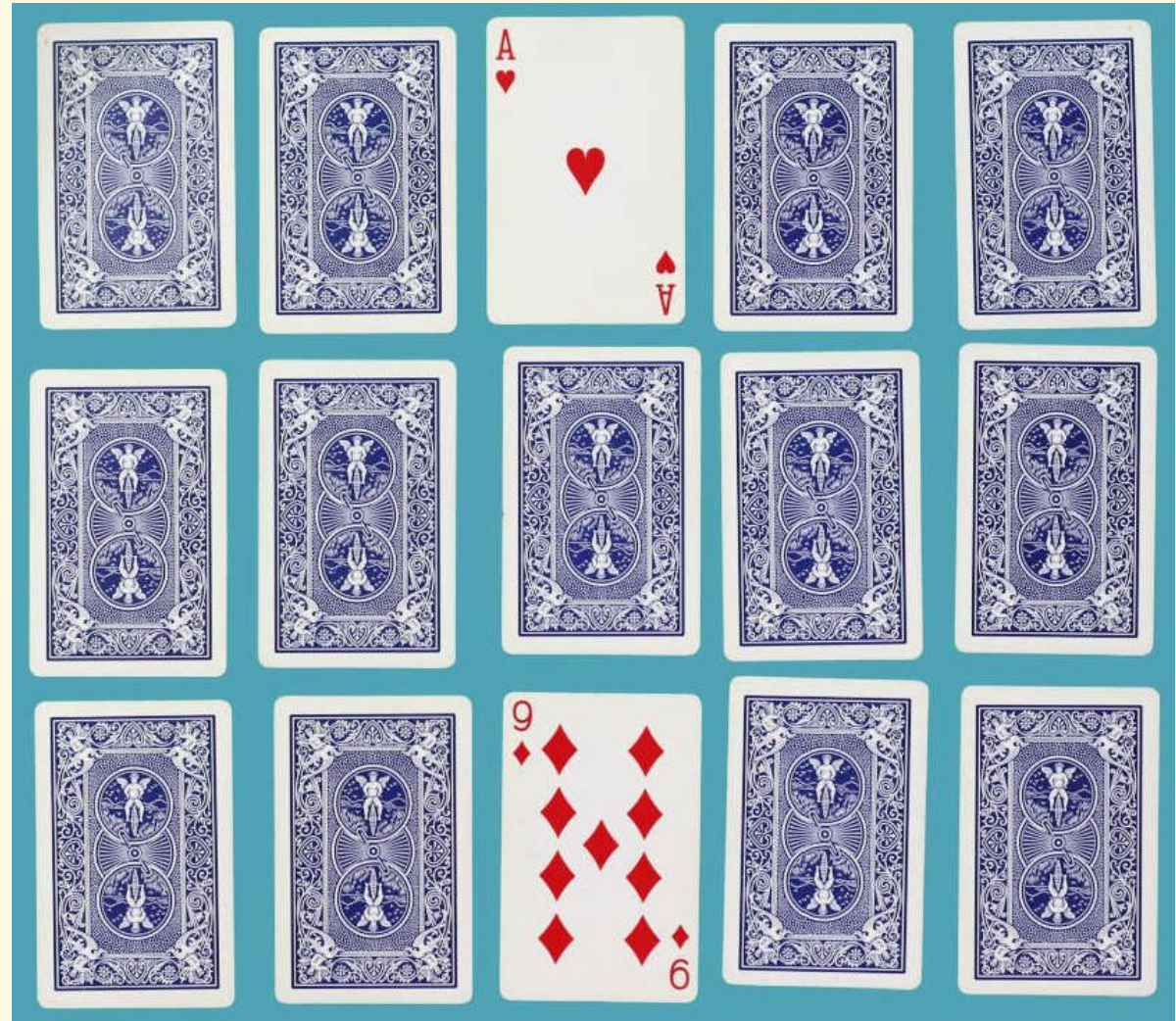
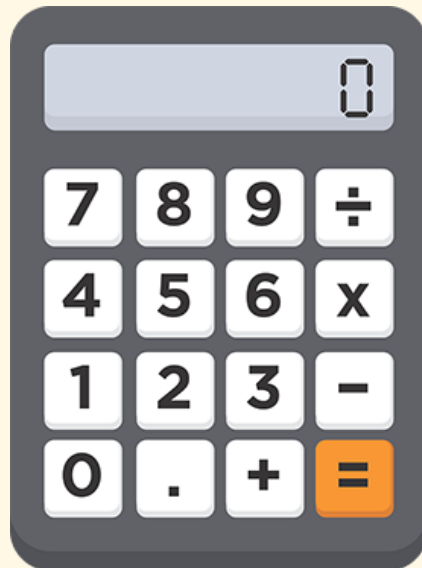
Enter height (cm):

Your BMI: 23.87

Status: Normal

Exercises

1. Write a memory card matching game
2. Create a simple calculator as in the figure



Communication with Server

Introduction

- ▶ React is purely for front-end development
- ▶ Two problems raise for a website built with React:
 - ▶ For deployment, a web server is required to host it
 - ▶ It occasionally needs to communicate with the server
- ▶ Web URLs to answering requests from the client are often called API endpoints
 - ▶ It can be easily created with Express.JS
 - ▶ Different ways to encapsulate communication data:
 - ▶ Using GET query parameters
 - ▶ Using POST body
 - ▶ Popular formats: plain text, JSON, XML,...